Explicación de los Flujos

Procesos de negocio y flujos técnicos del módulo administrativo

Explica, paso a paso y con diagramas, cómo fluyen la información, el stock, el dinero y el código en FarmaClick.

Proyecto	FarmaClick - Sistema de gestión para farmacias - Módulo administrativo
Cliente / contraparte	MediFarm
Equipo	D-Software: Alejandro Fuentes Navarrete, Víctor Rubilar Inostroza, Robert González Muñoz
Asignatura	PTY4614 - Capstone
Metodología	Scrum (sprints de 2 semanas)
Código del documento	DOC-10
Versión	1.1
Fecha	21 de octubre de 2026
Repositorio	https://github.com/Alefuentes982/FarmaClick

Control de versiones

Versión	Fecha	Autor	Cambios
1.0	16 de octubre de 2026	Alejandro Fuentes (PO)	Versión inicial, sobre el Incremento 4.
1.1	21 de octubre de 2026	Alejandro Fuentes (PO)	Revisión de redacción y formato.

Contenido

1. Mapa de flujos 3
2. Abastecimiento: de la orden de compra al stock 4
3. Venta presencial en caja 6
4. Ciclo de vida del pedido 8
5. Devolución 9
6. Cuadratura y cierre de caja 11
7. Una solicitud a la API 14
8. Movimiento del stock dentro de la base 15
9. Del código a producción (CI/CD) 16
10. Cambios de base de datos 17
11. Operación 18

1. Mapa de flujos

Flujo	Tipo	Diagrama	Sección
Abastecimiento: de la orden de compra al stock	Negocio	Actividad + secuencia + estados	2
Venta presencial en caja	Negocio	Actividad + secuencia	3
Ciclo de vida del pedido	Negocio	Estados	4
Devolución	Negocio	Actividad	5
Cuadratura y cierre de caja	Negocio	Actividad + estados + secuencia	6
Una solicitud a la API (autenticación y autorización)	Técnico	Secuencia	7
Movimiento del stock dentro de la base	Técnico (datos)	Tabla	8
Del código a producción (CI/CD)	Técnico (DevOps)	Flujo CI/CD	9
Cambios de base de datos (migraciones)	Técnico	Pasos	10
Operación: mantener activo, respaldo y certificado	Técnico	Operación	11

2. Abastecimiento: de la orden de compra al stock

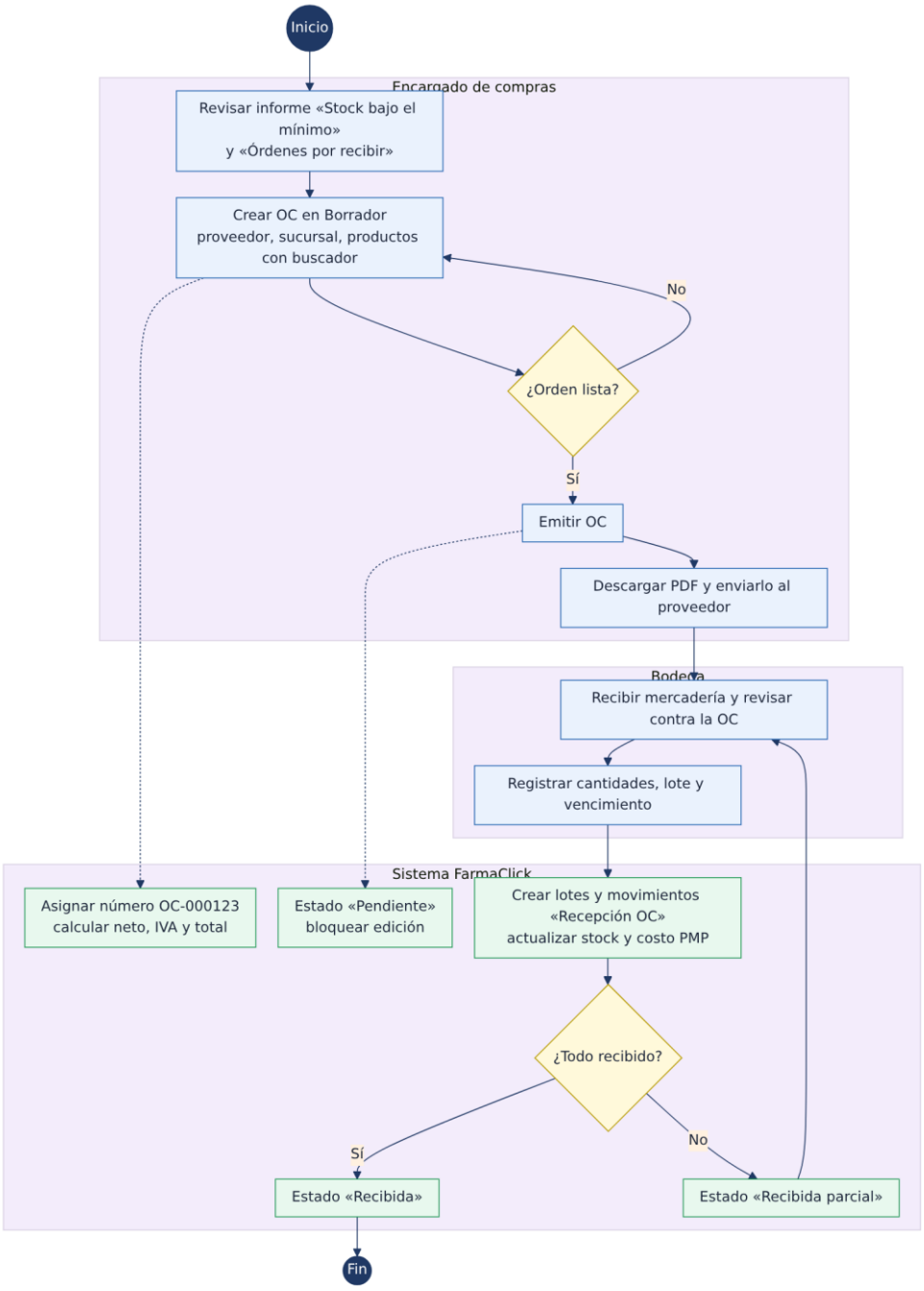


Figura 1. Actividad: abastecimiento (carriles: encargado de compras, sistema, bodega).

1. El encargado de compras revisa los informes **Stock bajo el mínimo** y **Órdenes por recibir** del dashboard.
2. Crea la OC en **Borrador**: elige proveedor y sucursal, y agrega productos con el buscador (SKU o nombre). La base asigna el número **OC-000NNN** y calcula neto, IVA y total con triggers.
3. Al **emitirla** pasa a **Pendiente** y ya no se puede editar; se descarga el PDF y se envía al proveedor.
4. Cuando llega la mercadería, bodega registra por línea la **cantidad recibida, el lote y el vencimiento**.

5. Por cada línea la API crea el **lote** y un movimiento **"Recepción OC"** en el kardex. Los triggers recalculan el **costo promedio ponderado (PMP)** del producto y suman el stock.
6. Si se recibió todo, la OC queda **Recibida**; si no, **Recibida parcial** y puede recibirse el resto después.

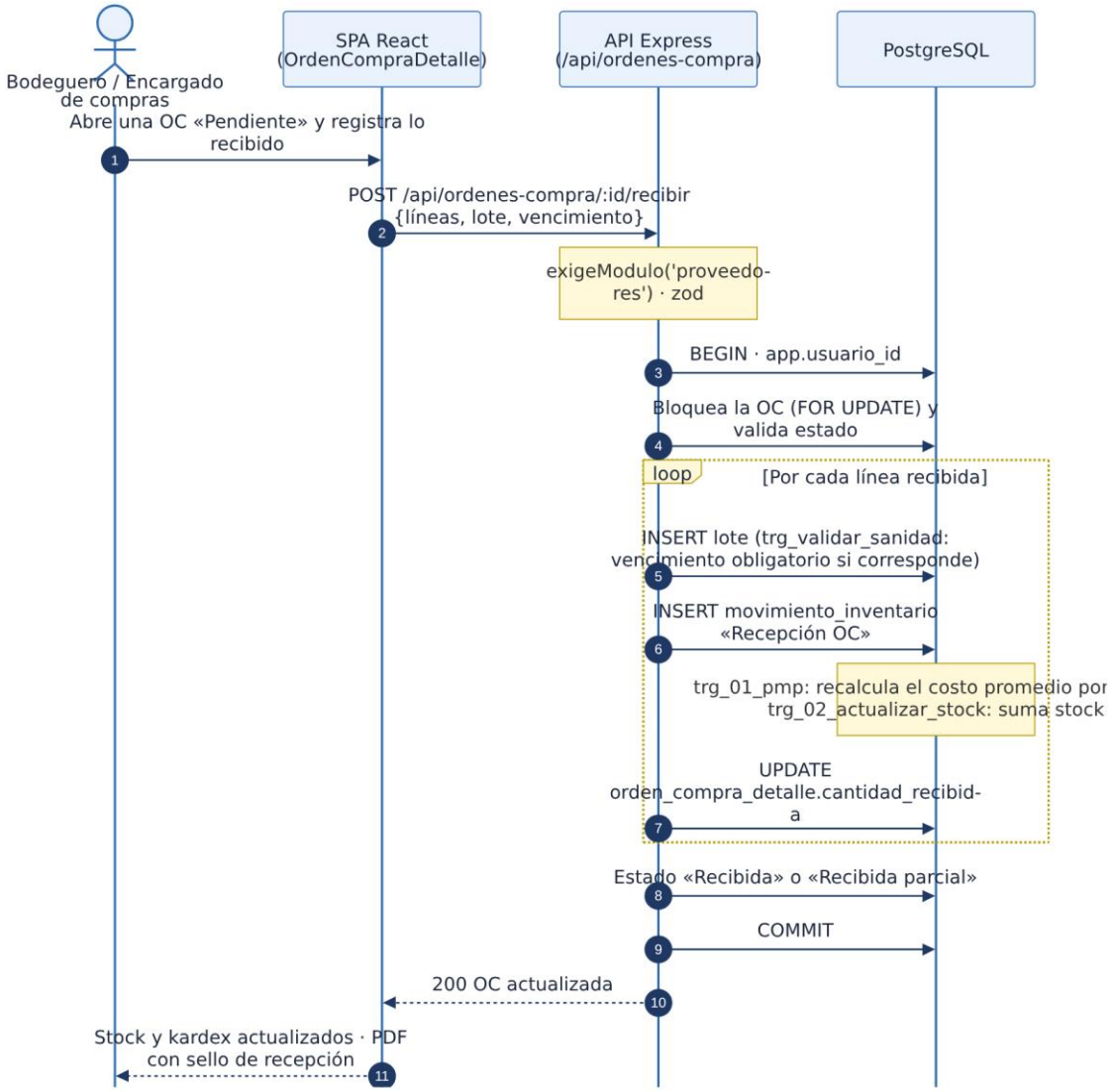


Figura 2. Secuencia: recepción de una orden de compra.

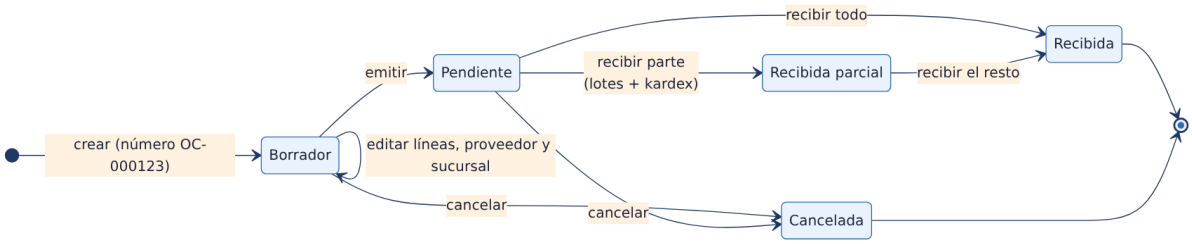


Figura 3. Estados de la orden de compra.

3. Venta presencial en caja

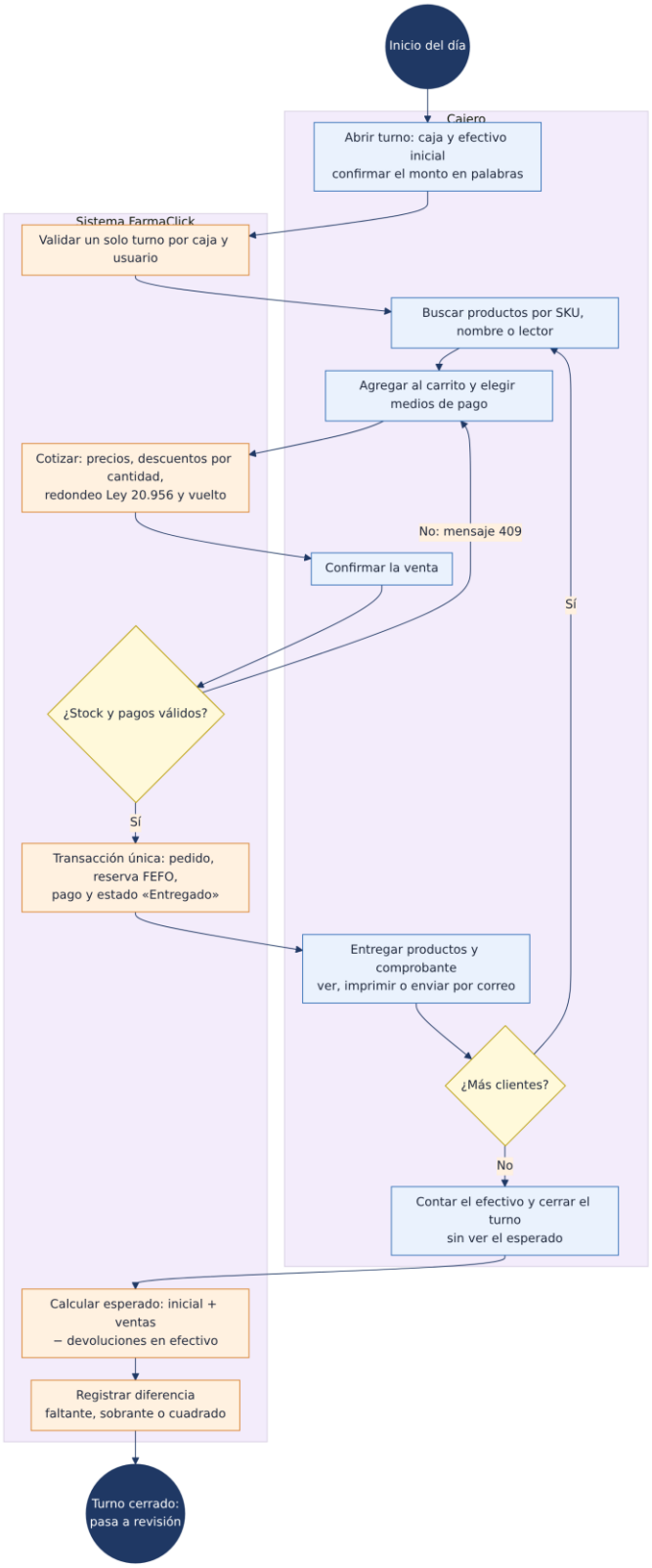


Figura 4. Actividad: venta presencial (carriles: cajero y sistema).

1. **Abrir turno:** el cajero elige su caja e informa el efectivo inicial; el monto se muestra **en palabras** para detectar errores de digitación. El sistema valida que no haya otro turno abierto en esa caja ni del mismo usuario.
2. **Buscar y armar el carrito** por SKU, nombre o lector de código de barras.
3. **Cotizar:** la API calcula precios y descuentos por cantidad (los fija la base), el **redondeo del efectivo** según la Ley 20.956 (1-4 baja, 5-9 sube) y el vuelto.
4. **Confirmar:** `POST /api/pos/ventas` ejecuta en **una sola transacción**: crea el pedido, reserva el stock por **FEFO** (primero vence, primero sale) generando movimientos "Venta", registra los pagos y marca el pedido **Entregado**. Si algo falla (stock, pagos que no cuadran), se revierte todo y se muestra el motivo (HTTP 409).
5. **Comprobante:** ver, descargar, imprimir o enviar por correo.
6. Al final del día, **cierre ciego**: el cajero cuenta el efectivo y lo informa sin ver el esperado; el sistema calcula **esperado = apertura + ventas en efectivo - devoluciones en efectivo** y la diferencia.

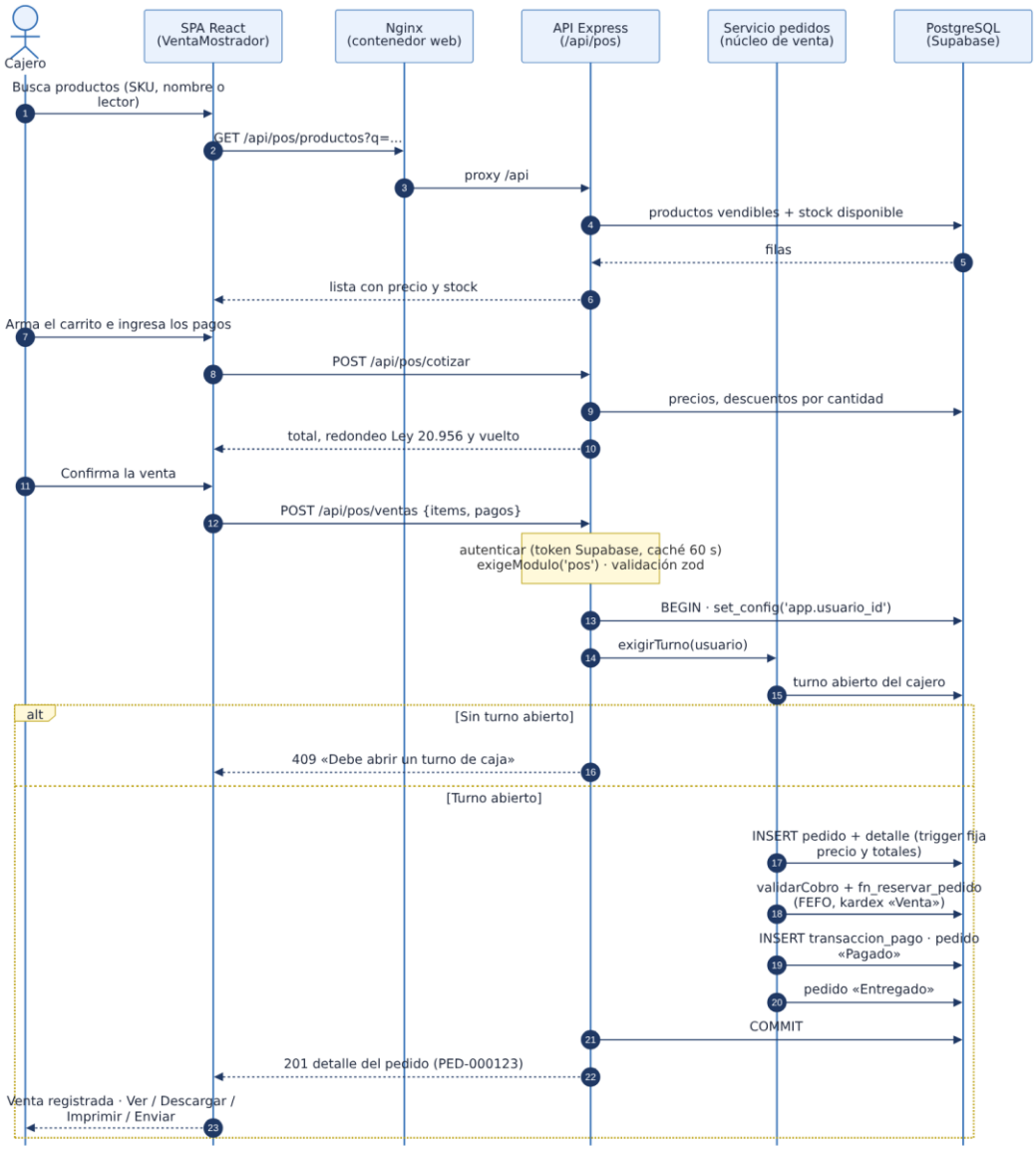


Figura 5. Secuencia: venta en mostrador.

4. Ciclo de vida del pedido

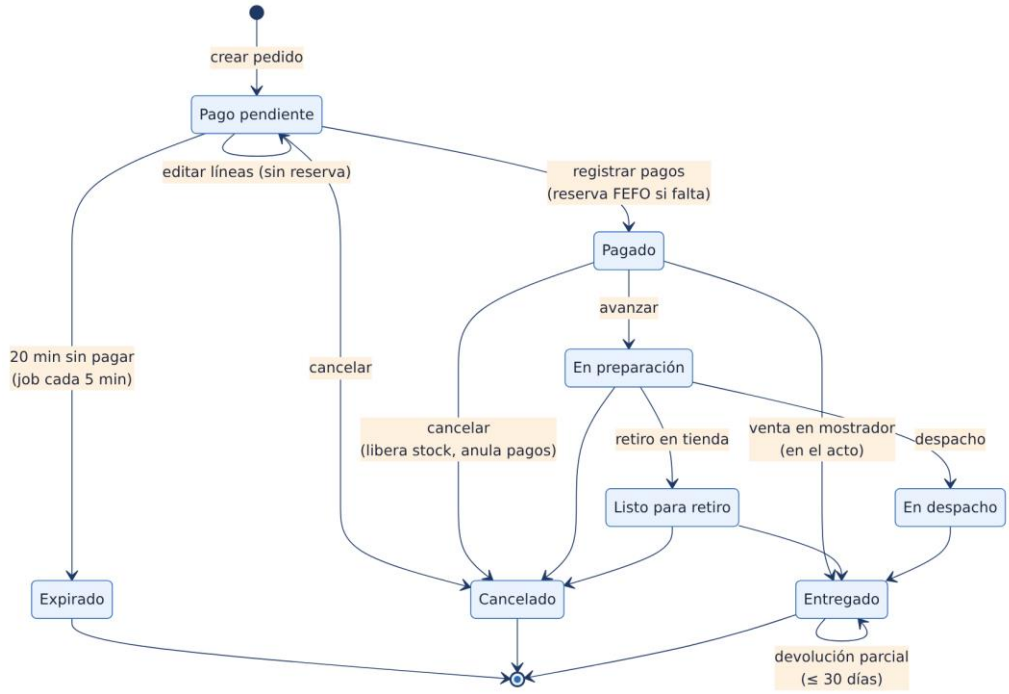


Figura 6. Estados del pedido.

- Un pedido nace en **Pago pendiente**; solo en ese estado y sin reserva se editan sus líneas.
- Al registrar los pagos se reserva el stock (si no estaba) y pasa a **Pagado**. En caja pasa de inmediato a **Entregado**.
- Los pedidos a distancia avanzan: **En preparación > Listo para retiro / En despacho > Entregado**.
- Un pedido sin pagar **expira a los 20 minutos** (job de pg_cron cada 5 minutos) y libera su stock.
- Se puede **cancelar** hasta "Listo para retiro": se devuelve el stock y se anulan los pagos.

5. Devolución

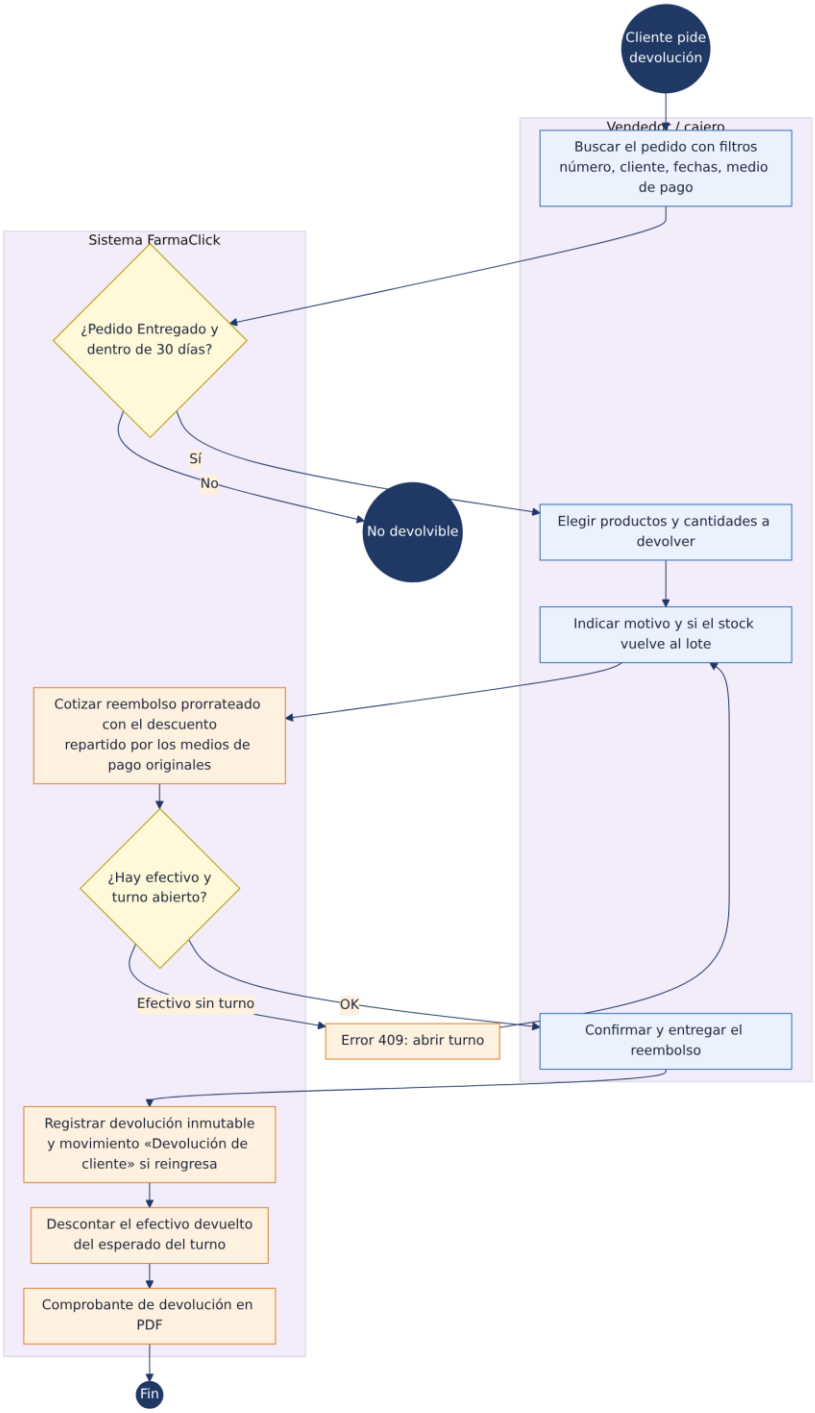


Figura 7. Actividad: devolución.

1. El vendedor busca el pedido con filtros (número, cliente, fechas, medio de pago). Solo aparecen pedidos **Entregados** dentro de `DEVOLUCION_MAX_DIAS` (30).
2. Elige productos y cantidades, el motivo y si el stock **vuelve al lote de origen** (por defecto no).
3. El sistema cotiza el reembolso **prorrateado con el descuento** de la venta y lo reparte por los **mismos medios de pago** originales; el efectivo se redondea y nunca supera lo pagado en efectivo.

4. Si hay efectivo, exige un **turno abierto**: ese efectivo se descuenta del esperado del cierre.
5. La devolución queda **inmutable**, genera movimientos "Devolución de cliente" si reingresa stock y su comprobante en PDF.

6. Cuadratura y cierre de caja

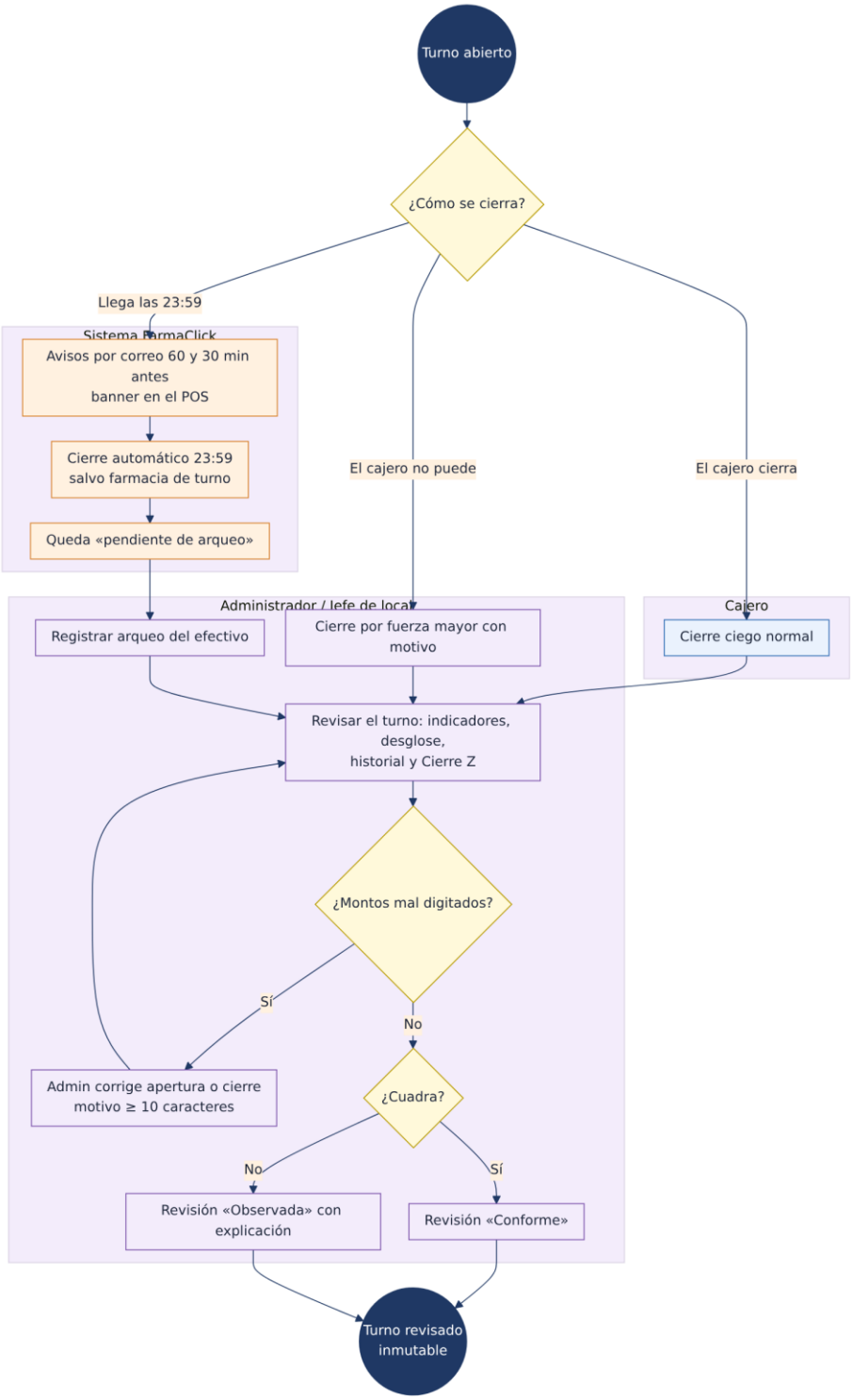


Figura 8. Actividad: cierre y revisión del turno.

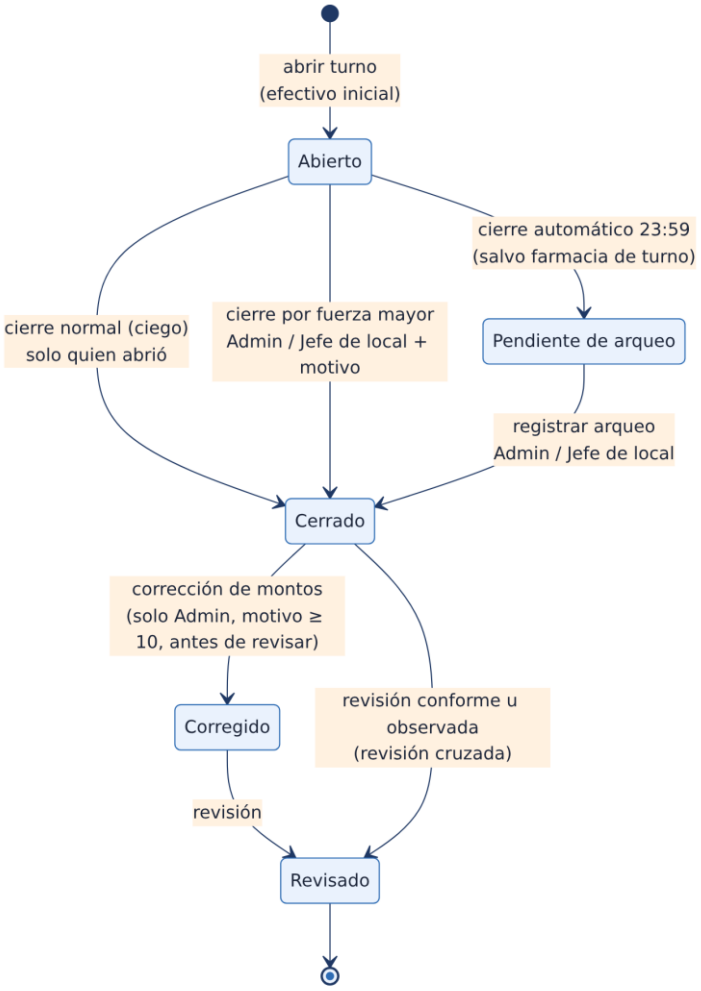


Figura 9. Estados del turno de caja.

Situación	Quién	Qué pasa
Cierre normal (ciego)	El cajero que abrió	Informa el contado; queda Cerrado con su diferencia.
Cierre por fuerza mayor	Administrador o Jefe de local (de esa sucursal)	Cierra el turno de otro con monto y motivo; cierre_tipo "forzado".
Cierre automático 23:59	pg_cron (59 * * * *) y la API cada minuto	Turnos abiertos (salvo farmacias de turno) quedan con cierre_tipo "automatico" y pendientes de arqueo.
Avisos	Tareas de la API + SMTP	Correos 60 y 30 min antes y aviso de cierre automático al cajero, Administradores y Jefes de local; una sola vez (turno_aviso).
Arqueo	Administrador o Jefe de local	Registra el efectivo contado de un turno pendiente.
Corrección de montos	Solo el Administrador	Antes de revisar, con motivo >= 10 caracteres; recalcula el esperado y queda en turno_correccion.
Observaciones	Cualquiera con acceso a cuadratura	Notas inmutables en cualquier estado.
Revisión	Otra persona (el Admin y el Jefe pueden revisar el suyo)	Conforme u Observada (con explicación). No se revisa un turno con arqueo pendiente.

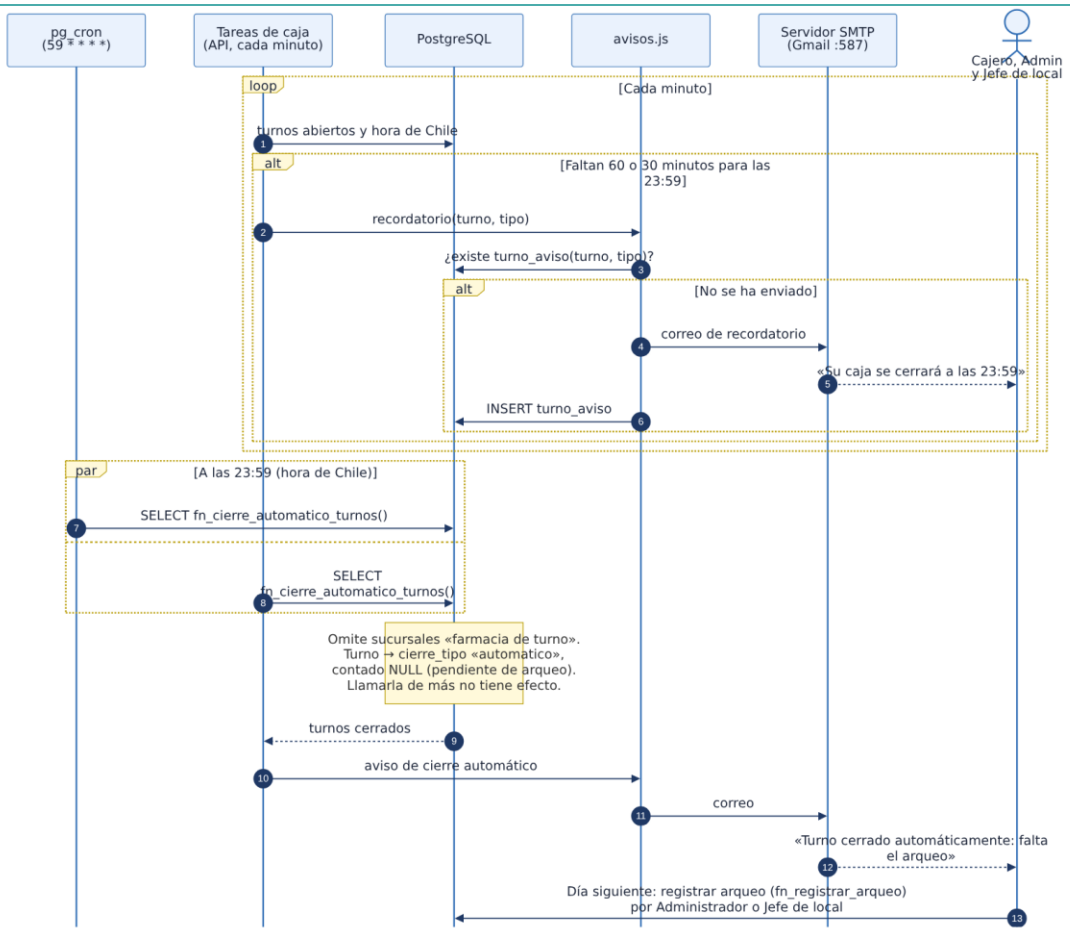


Figura 10. Secuencia: cierre automático y avisos.

7. Una solicitud a la API

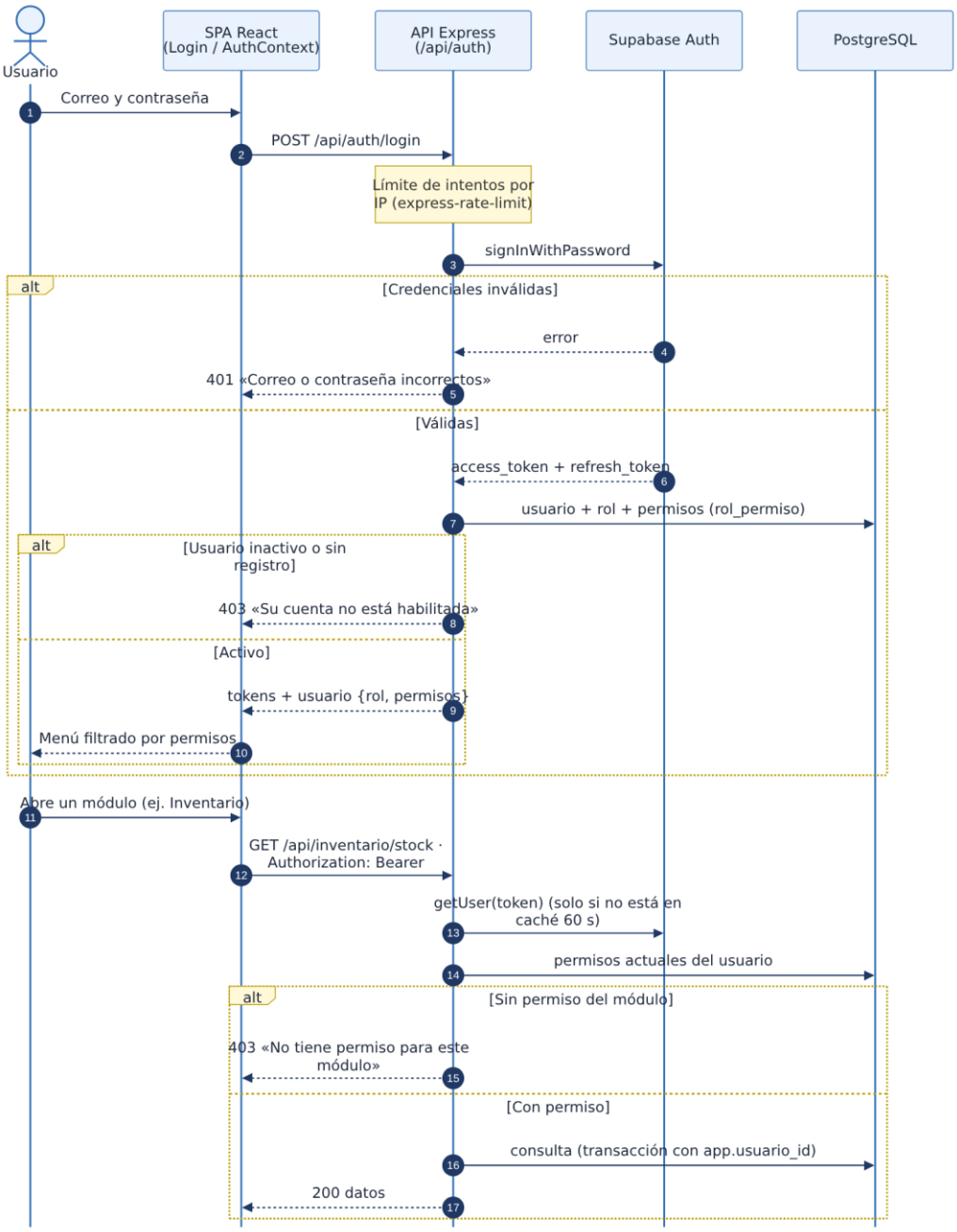


Figura 11. Secuencia: inicio de sesión y autorización.

1. El navegador llama a `/api/...` en el mismo dominio; **Nginx** reenvía la solicitud a la API (en producción, antes pasa por el **balanceador** que termina el HTTPS).
2. **autenticar:** valida el token Bearer con Supabase Auth (resultado en caché 60 s) y carga el usuario, su rol y sus **permisos actuales** desde la base.
3. **exigeModulo:** verifica el permiso del módulo (o un permiso de lectura cruzada para GET). Sin permiso > 403.
4. **Validación:** el cuerpo y los filtros se validan con **zod** (`esquemas.js`). Datos inválidos > 400.
5. **withTx:** abre una transacción y fija `app.usuario_id`, que las funciones y triggers usan para firmar cada movimiento.

6. La **base** aplica las reglas (triggers y funciones). Una regla violada lanza `RAISE EXCEPTION` > la API responde **409** con la explicación.
7. La respuesta es JSON; los PDF, Excel y CSV viajan como **base64 dentro de JSON** y el frontend los convierte en archivo.

8. Movimiento del stock dentro de la base

Evento	Movimiento de kardex (motivo)	Efecto
Recepción de OC	Entrada - Recepción OC	Crea lote; suma stock; recalcula PMP
Reserva / venta	Salida - Venta (por lote FEFO)	Descuenta stock del lote que vence primero
Cancelación o expiración	Entrada - Liberación de reserva	Devuelve el stock reservado
Devolución con reingreso	Entrada - Devolución de cliente	Vuelve al lote de origen
Ajuste positivo / negativo	Entrada / Salida - Ajuste	Con observación obligatoria
Merma	Salida - Merma	Con observación obligatoria
Lote vencido	Salida - Producto vencido	Manual o job diario 05:00 UTC

Todos los movimientos son **inmutables** (trigger `prevent_kardex_upd`): una corrección es siempre un nuevo movimiento. El stock de cada lote y el `stock_total_web` del producto los mantienen los triggers, nunca la aplicación.

9. Del código a producción (CI/CD)

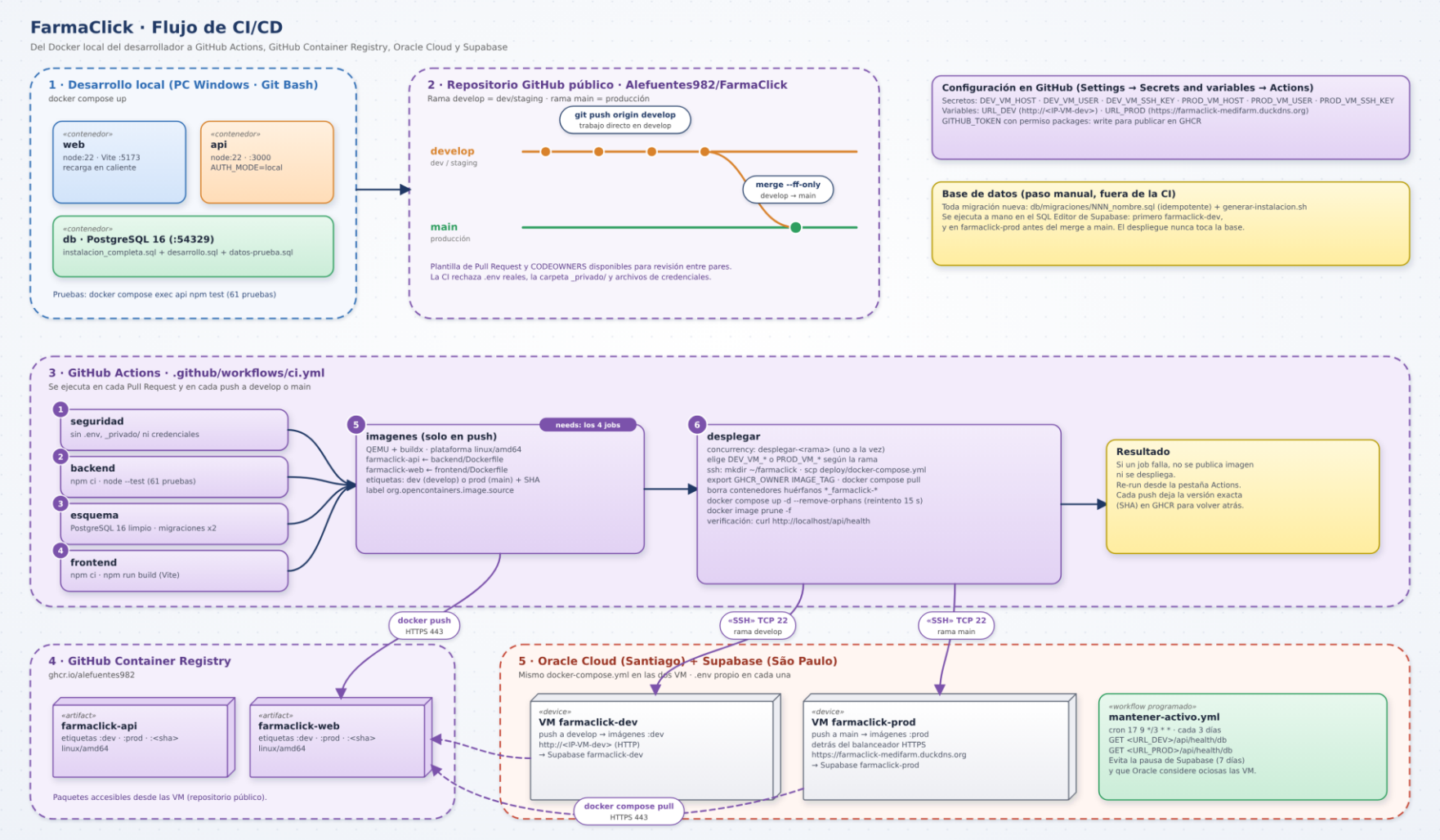


Figura 12. Flujo de CI/CD.

1. **Local:** el desarrollador programa y prueba con `docker compose up` (base, API y web en contenedores) y ejecuta `npm test`.
2. **Subir los cambios a develop:** `git push origin develop`.
3. **CI (GitHub Actions):** `seguridad` (sin secretos), `backend` (61 pruebas), `esquema` (instalación y migraciones sobre PostgreSQL limpio) y `frontend` (build). Si alguno falla, el flujo se detiene.
4. **Imágenes:** se construyen `farmaclick-api` y `farmaclick-web` para **linux/amd64** y se publican en **GHCR** con la etiqueta `dev` y el SHA.
5. **Despliegue en dev:** por SSH a la VM de dev: copia `docker-compose.yml`, `docker compose pull`, limpia contenedores huérfanos, `up -d` y verifica `/api/health`. Un despliegue a la vez por entorno.
6. **Prueba en dev:** el equipo y el PO prueban en `http://<IP-VM-dev>` contra Supabase dev.
7. **Paso a producción:** migraciones en Supabase prod, `git merge --ff-only origin/develop` en main y push. El mismo flujo publica la etiqueta `prod` y despliega en la VM de producción detrás del balanceador HTTPS.
8. **Verificación:** prueba de humo en `https://farmaclick-medifarm.duckdns.org`.

10. Cambios de base de datos

1. Crear `db/migraciones/NNN_nombre.sql` (número siguiente), idempotente y dentro de `BEGIN/COMMIT`.
2. Regenerar la instalación: `bash db/generar-instalacion.sh`.
3. Probar en local: `docker compose down -v && docker compose up` (la base se crea con la instalación completa).
4. Push a develop: la CI valida que se instala y que se puede repetir.
5. Ejecutar la migración en el SQL Editor de **Supabase dev**; probar en dev.
6. Antes del merge a main, ejecutarla en **Supabase prod**.

11. Operación

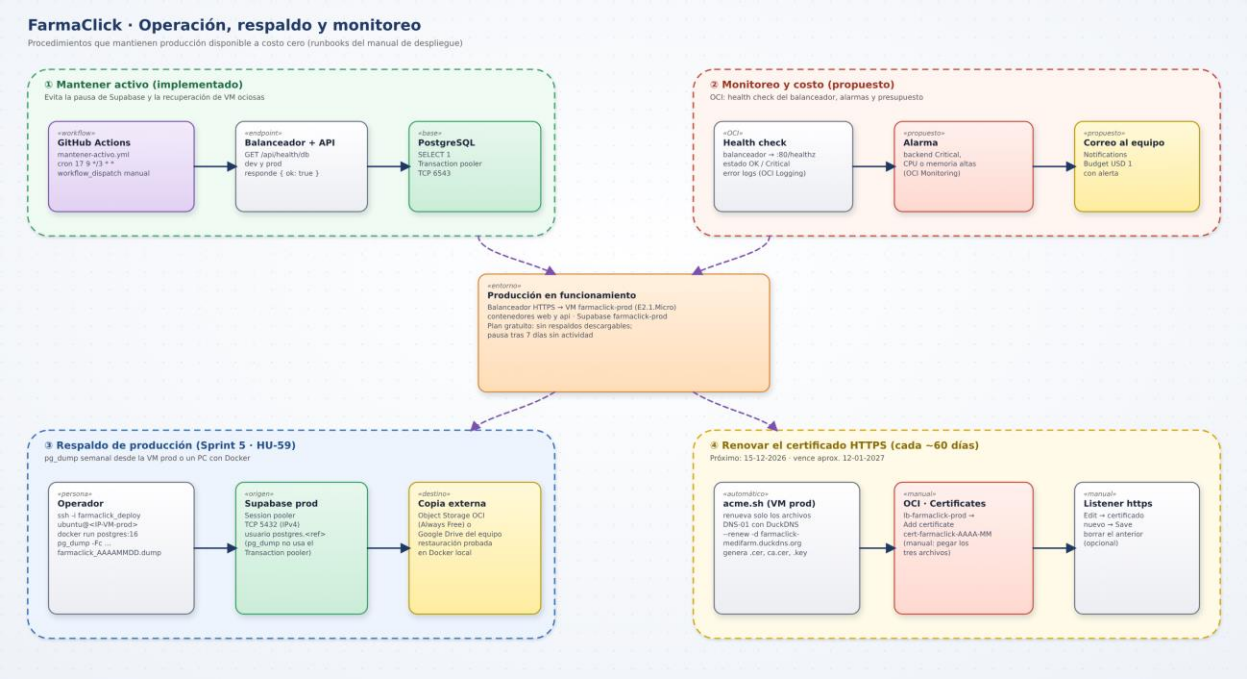


Figura 13. Mantener activo, monitoreo, respaldo y renovación del certificado.

- **Mantener activo:** cada 3 días GitHub Actions consulta `/api/health/db` en dev y prod; evita la pausa de Supabase (7 días) y que Oracle considere ociosas las VM.
- **Monitoreo:** el balanceador revisa `/healthz`; se propone agregar alarmas de OCI Monitoring y un presupuesto de USD 1.
- **Respaldo:** `pg_dump` semanal de producción por el Session pooler, guardado fuera de Supabase (HU-59).
- **Certificado:** acme.sh renueva los archivos; el certificado se vuelve a cargar a mano en el balanceador cada ~60 días (próximo 15-12-2026).